Q-1 Mobile Computing Architecture: 3-Tier Architecture in Modern Mobile Apps

-> The 3-tier architecture in mobile applications separates concerns across three layers:

- Presentation Tier (Client UI):

This is the user interface of the mobile app (e.g., screens and activities). It handles user interaction and input/output. It doesn't process business logic or data storage directly but interacts with the Application Tier via APIs

- Application Tier (Business Logic)

This tier is typically hosted on a backend server. It process inputs, enforces rules, performs calculations, and makes decisions. It acts as the intermediary between the UI and data tiers.

- Data Tier (Storage):

This tier consists of databases (e.g., SQL/NoSQL) or persist storage. It stores user data, application metadata, and other necessary information.

* Example (User Login):

1. User enters credentials on the UI (Presentation Tier).
2. The app sends a request to the backend (Application Tier).
3. The backend verifies credentials against stored data in a database (Data Tier)
4. The response is sent back to the Presentation Tier with login success or failure.

Q-2 Core Design Principles for Mobile

-> 1. Limited Resources (CPU, Memory, Battery):
-> Mobile apps must optimize performance to avoid draining resources. Efficient code and background work management are crucial

2. Varying Screen Sizes and Densities:
-> Apps must be responsive and use flexible layouts and scalable units (dp/sp) to ensure consistent UX across devices.

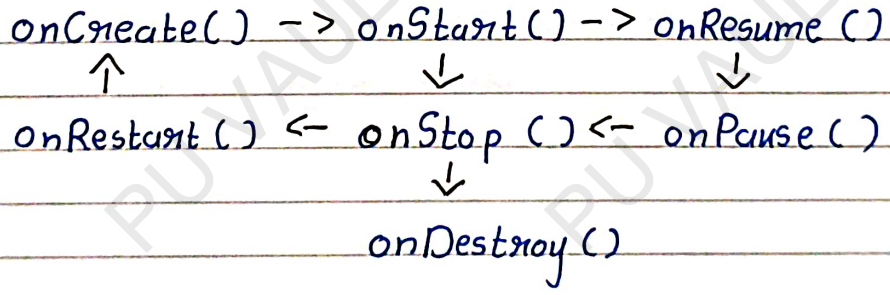
3. Touch-Centric Interaction:
-> Unlike desktops, mobile interfaces rely on touch gestures. This requires larger tappable areas and gesture-based UI logic.

4. Intermittent Connectivity:
-> Apps should handle offline scenarios gracefully by caching data locally and syncing when online.

5. Security and Privacy:
-> Mobile devices are personal, so secure data transmission (HTTPS), authentication, and permissions management are essential in design.

Q-3 The Android Activity Lifecycle

* State Diagram Overview:



* Key Lifecycle Callbacks :

- onCreate() - initialize UI, set up resources
- onStart() - app becoming visible
- onResume() - app is interactive
- onPause() - partially obscured
- onStop() - app no longer visible
- onDestroy() - cleanup before destruction
- onRestart() - returning to foreground after stop

* Why stop heavy operations in onStop() ?

-> onPause() is brief and meant for lightweight tasks like saving small UI states. onStop() is more reliable for releasing large resource because it confirms the activity is no longer visible.

Q-4 A Comparison of UI Layouts

Layout	Description	Pros	Cons	Best Use
Linear Layout	Arranges views in a row or column	Simple, Predictable	Nested layouts can reduce Performance	Simple forms, Vertical lists
Relative Layout	Positions views relative to each other	More flexible than Linear Layout	Complex relationships can be hard to manage	Dialogs, moderate complexity Layouts
Constraint Layout	Uses constraints to position views flexibly	Powerful, flat layout hierarchy	Steeper learning curve	Complex UIs with performance in mind

Q-5 Background Processing Concepts

Paradigm	Use Case	Lifecycle	Tied to UI Thread?
Thread	Parallel processing for custom tasks	Manual control; dies with activity	Yes (unless explicitly detached)
AsyncTask	Quick background work with UI updates	Short-lived, auto-managed	Runs in background, results on UI thread
Service	Long-running background operations	Independent, can run even when app is not visible	Runs on main unless separate thread used

*Summary :-

Use Threads for short, manual control tasks. Avoid AsyncTask in modern apps. Use Services for long-running or scheduled tasks.

Q-6 Android's Component Model & Communication via Intents

-> The Intent object is a messaging tool used to request actions from other components.

- Explicit Intent :

-> Directly specifies the target component.
Use case : Start a known activity within the app.

* Code :-

```
Intent intent = new Intent(this, LoginActivity.class);
startActivity(intent);
```

- Implicit Intent :

-> Declares a general action to be performed, allowing the system to find a component.
Use case : Open a URL in any browser.

* Code :-

```
Intent intent = new Intent(Intent.ACTION_VIEW,
                Uri.parse("https://pu-vault.vercel.app"));
startActivity(intent);
```